

Дилема подвійного агента

- **Обмеження часу:** 12 хвилин.
- **Сховище:** 5 GB
- **Середовище:** один GPU (≈ 16 GB VRAM), без інтернету
- **Розмір розв'язку:** `solution.ipynb` ≤ 1 MB
- **Базовий результат:** 0

У національному центрі штучного інтелекту в Астані дві комп'ютерні моделі — Model R (ResNet-18) і Model V (ViT-Tiny) — аналізують фотографії. Наразі обидві моделі працюють бездоганно, досягаючи точності 100% і погоджуючись щодо кожного зображення. Щоб перевірити, наскільки насправді відрізняються їхні розумні «мізки», головний науковець ставить перед вами завдання: внести до кожної фотографії крихітні, майже непомітні зміни пікселів так, щоб Model R і Model V повністю розійшлися у своїх відповідях.



1. Завдання

Два попередньо навчені класифікатори зображень аналізують те саме зображення. На зображеннях, наданих у цьому завданні, обидва класифікатори працюють із точністю 100%.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (Transformer, ViT-Tiny).

Ваше завдання — створити невелику зміну («збурення») для кожного зображення так, щоб дві моделі не погоджувалися між собою. Для кожного зображення ви повинні створити **два різні** збурення:

- **Тип А:** після його додавання Model R усе ще класифікує зображення правильно, але Model V класифікує його неправильно.
- **Тип В:** після його додавання Model V усе ще класифікує зображення правильно, але Model R класифікує його неправильно.

Кожне збурення має бути достатньо *малим*, щоб його було важко помітити. Менші збурення дають вищий результат (див. Розділ 5). Збурення застосовується безпосередньо до оригінального зображення на рівні пікселів.

2. Загальнодоступні дані

Разом із завданням надається набір зображень, організований у дві вибірки — `train` (100 зображень) і `test_public` (100 зображень), — кожна з яких містить зображення різної роздільної здатності. Усі зображення належать до 1000 класів ImageNet-1K, і як Model R, так і Model V досягають точності 100% на обох вибірках.

Надаються такі файли:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Під час оцінювання ваша папка `dataset/test_public/` прозоро замінюється двома прихованими наборами зображень (`test_leaderboard_a` і `test_leaderboard_b`) для офіційного оцінювання. Кожен із них містить **100 зображень** у форматі PNG і файл міток.

Примітка: у цьому завданні мітки в тестових датасетах доступні.

3. Формат виведення

Для кожного зображення ви повинні створити два файли:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), відповідає назві зображення в датасетах.
- Кожен файл є одним тензором, збереженим за допомогою `torch.save`. Його форма має бути $3 \times H \times W$, де H і W відповідають **оригінальній** роздільній здатності цього

зображення (не 224×224).

- Код має створювати лише один ZIP-файл, `submission.zip`. Розмістіть усі файли `.pt` на верхньому рівні ZIP-архіву, без папки-контейнера чи підкаталогів.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Ноутбук попередить вас, якщо у форматі виведення будуть будь-які проблеми.

4. Обмеження

- **Моделі:** ви повинні використовувати `torchvision.models.resnet18(pretrained=True)` і `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Жодні інші попередньо навчені моделі не дозволені.
- **Конвеєр перетворень (застосовується під час оцінювання):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` для подробиць.
- **Роздільна здатність збурення:** має відповідати **оригінальній** роздільній здатності необробленого зображення (не 224×224). Тензор додається до необробленого зображення до застосування конвеєра перетворень.
- **Формат виведення:** лише файли `.pt` — жодних PNG/JPG. Тензори додаються до необробленого зображення, а значення пікселів перед попередньою обробкою обмежуються діапазоном `[0, 1]`.
- **Назви файлів:** плоский список, строгий формат `{index}_a.pt / {index}_b.pt`. Жодних підкаталогів усередині zip.
- **Бібліотеки:** `torch`, `torchvision`, `timm`.

5. Оцінювання

Підсумковий результат обчислюється так. Нехай m — кількість зображень у вибірці, $Score_A$ — кількість успішних збурень Типу A, а $Score_B$ — кількість успішних збурень Типу B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF — це функція, призначена для штрафування збурень із високою нормою та дуже чутлива поблизу верхньої межі результативності. Вона обмежена діапазоном від 0.5 до 1. Повну реалізацію можна переглянути в Розділі 8 файлу `solution.ipynb`.

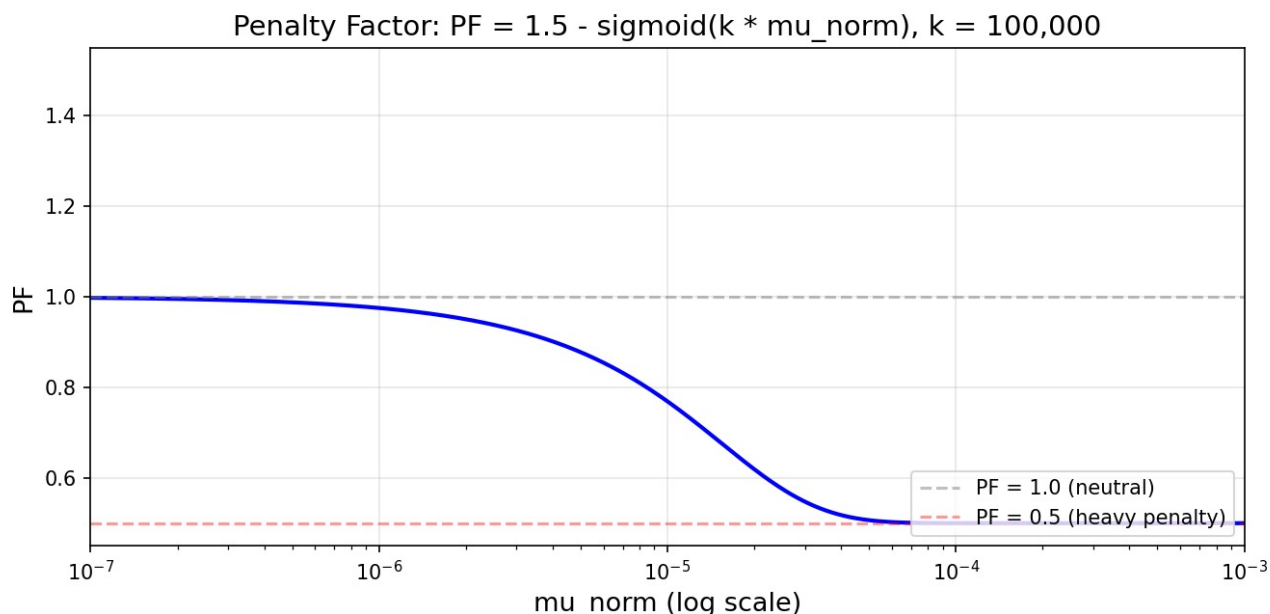


Рисунок: крива штрафної функції.

6. Перевірка подання

У ноутбучі є перевірки, які попередять вас про проблеми з форматуванням, у Розділі 7 ноутбука `solution.ipynb`.

7. Локальне тестування

`solution.ipynb` містить повний робочий приклад. Він завантажує загальнодоступні дані, обидві моделі й офіційний засіб оцінювання та створює ZIP-файл подання. Прочитайте його перед початком роботи.

8. Як подати розв'язок

- Збережіть свої зміни у `solution.ipynb`.
- Відкрийте вкладку Git на лівій бічній панелі JupyterLab.
- **Додайте до індексу** `solution.ipynb` (піктограма + поруч із ним).
- Введіть повідомлення коміту й натисніть **Commit**.
- Натисніть піктограму хмари зі стрілкою вгору, щоб виконати push.
- Поверніться на цю сторінку змагання й натисніть **Submit**.

Подайте рівно один файл із назвою `solution.ipynb`.